Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

ЗВІТ

з лабораторної роботи № 8
з дисципліни «Основи програмування»
«Масиви»

Виконав:
студент групи ФЕП-13
Марцінко Н. Р.

Перевірив:
асистент Васюта В. М.

Львів – 2026

Мета роботи

Вивчити поняття і застосування масивів.

Хід роботи

1. Функція заповнення масиву з 10 випадкових чисел [0…99]

Написано функцію fill_random_array(int arr[], int size), яка в циклі записує в кожен елемент значення rand() % 100. У main оголошено масив array[10], викликано srand(time(NULL)) для ініціалізації генератора, заповнено масив і виведено його на екран.

Martsinko > Lab8 > c task1.c

```c
#include<stdio.h>
#include<stdlib.h>
#include<time.h>


void fill_random_array(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] = rand() % 100;
    }
}

int main(){
    int array[10];
    srand(time(NULL));
    fill_random_array(array, 10);
    printf("Згенерований масив:\n[ ");
    for (int i = 0; i < 10; i++) {
        printf("%d ", array[i]);
    }
    printf("]");


    return 0;
}
```

Рис. 1. Код: fill_random_array і вивід згенерованого масиву

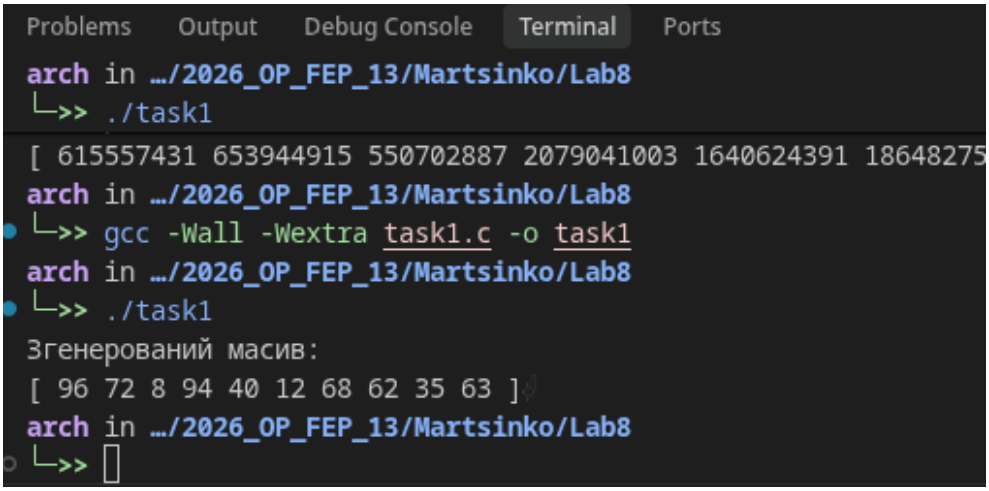


Рис. 2. Компіляція (gcc -Wall -Wextra) і результат: [96 72 8 94 40 12 68 62 35 63]

Програма зібрана з попередженнями -Wall -Wextra і відпрацювала коректно: десять псевдовипадкових чисел у заданому діапазоні.

2. Пошук елемента за індексом (вказівник, 1D і 2D)

Функція get_element_1d(int arr[10], int index) перевіряє, чи індекс у межах 0…9. Якщо так — повертає &arr[index], інакше NULL. Аналогічно get_element_2d для масиву 12×12: перевіряє обидва індекси і повертає адресу елемента або NULL.

У main створено одновимірний масив {10, 20, …, 100} і двовимірний, у якому arr2D[5][3] = 42. Запит елемента з індексом 5 у 1D і (5, 3) у 2D повертає валідні адреси; запит з індексом 99 дає NULL — захист від виходу за межі.



Рис. 3. get_element_1d / get_element_2d: перевірка меж і повернення адреси

```c
int* get_element_2d(int arr[12][12], int index_1, int index_2) {
    return NULL;
}

int main(){
int arr1D[10] = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
    int arr2D[12][12] = {0};
    arr2D[5][3] = 42;

    int* element_1d = get_element_1d(arr1D, 5);
    if (element_1d != NULL) {
        printf("адреса: %p\n", (void*)element_1d);
    } else {
        printf("елемент не знайдено!\n");
    }

    int* element_2d = get_element_2d(arr2D, 5, 3);
    if (element_2d != NULL) {
        printf("адреса: %p\n", (void*)element_2d);
    } else {
        printf("елемент не знайдено!\n");
    }

    int* bad_ptr = get_element_1d(arr1D, 99);
    if (bad_ptr == NULL) {
        printf("NULL\n");
    }
    return 0;
}
```

View 1 edited file | task1.c Alt+L >

Рис. 4. Виклик з невалідним індексом 99 → NULL

```
└─>> ./task2
адреса: 0x7fffb7bb2734
адреса: 0x7fffb7bb284c
NULL
arch in …/2026_OP_FEP_13/Martsinko/Lab8
```

Рис. 5. Вивід: дві валідні адреси та NULL для індексу поза межами

Важливо не розіменовувати вказівник без перевірки на NULL — інакше програма впаде з Segmentation fault. Тут перевірка є, тому все відпрацьовує безпечно.

3. Мінімум, максимум і сума в масиві з 20 випадкових чисел

Функція find_min_max_sum приймає масив, розмір і два вказівники int *min, int *max. Спочатку в *min і *max записується arr[0], далі цикл з i = 1 оновлює мінімум і максимум. Функція повертає суму *min + *max (за умовою завдання — суму знайдених min і max, а не всіх елементів).

У main масив з 20 елементів заповнюється випадково, викликається find_min_max_sum(array, 20, &min, &max). Передача через & дозволяє функції змінити змінні min і max у main.

```c
#include <stdlib.h>
#include <time.h>

void fill_random_array(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] = rand() % 100;
    }
}

int find_min_max_sum(const int arr[], int size, int *min, int *max) {
    *min = arr[0];
    *max = arr[0];

    for (int i = 1; i < size; i++) {
        if (arr[i] < *min) {
            *min = arr[i];
        }
        if (arr[i] > *max) {
            *max = arr[i];
        }
    }

    return *min + *max;
}

int main(void) {
    srand(time(NULL));

    int array[20];
    fill_random_array(array, 20);

    int min, max;
    int sum = find_min_max_sum(array, 20, &min, &max);

    printf("Min: %d\n", min);
    printf("Max: %d\n", max);
    printf("Sum: %d\n", sum);

    return 0;
}
```

Рис. 6. find_min_max_sum: оновлення min/max через вказівники, повернення суми

```
Min: 3
Max: 97
Sum: 100
```

Рис. 7. Результат: Min: 3, Max: 97, Sum: 100

На цьому запуску мінімум 3, максимум 97, їхня сума 100 — усе збігається з логікою функції.

Висновок

У цій роботі нарешті стало зрозуміло, як комп'ютер насправді бачить масиви і пам'ять.

Найбільше мороки було з вказівниками. Спочатку постійно плутався між самою адресою комірки (ptr) і тим, що в ній реально лежить (*ptr). Особливо на практиці відчувся ризик зловити Segmentation fault: якщо функція повертає NULL через кривий індекс, а в main спробувати вивести його через зірочку без перевірки, програма просто падає. Також незручно, що масиви в C не знають власної довжини і доводиться всюди вручну тягати розмір окремою змінною.

Але загалом після сучасних низькорівневих мов C відчувається як відверто застарілий інструмент. Усі три завдання (випадковий масив, пошук елемента з NULL, min/max/sum) налагоджено, результати відповідають умовам.